University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

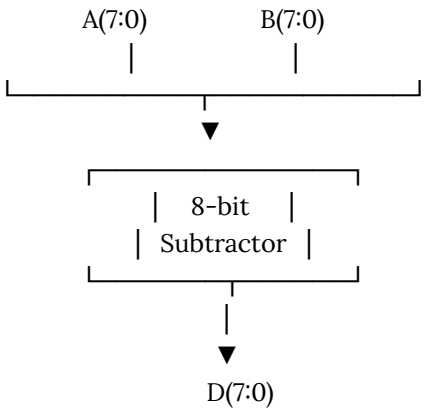
Mid-Term Lab Examination | Group C2 | Spring 2026

Duration	30 minutes	**Total Marks**	20
Student Name	_____	**ID**	_____

Mid-Term Task: 8-bit Subtractor

Objective. Design and implement an 8-bit subtractor in VHDL using the subtractor concepts developed in the laboratory. The circuit must perform the operation D = A − B, where A and B are 8-bit binary inputs and D is the 8-bit difference output.

System Architecture



Functional Requirement. For every applied pair of inputs, the circuit must produce the 8-bit difference D = A − B.

Required Test Cases

Use the following cases in your testbench and verify the expected 8-bit output.

Test	A	B	Operation	Expected D
1	00001111	00000011	15 − 3	00001100
2	00101010	00001010	42 − 10	00100000
3	01010101	00110011	85 − 51	00100010
4	11111111	00000001	255 − 1	11111110
5	10000000	00000001	128 − 1	01111111

Implementation Requirements

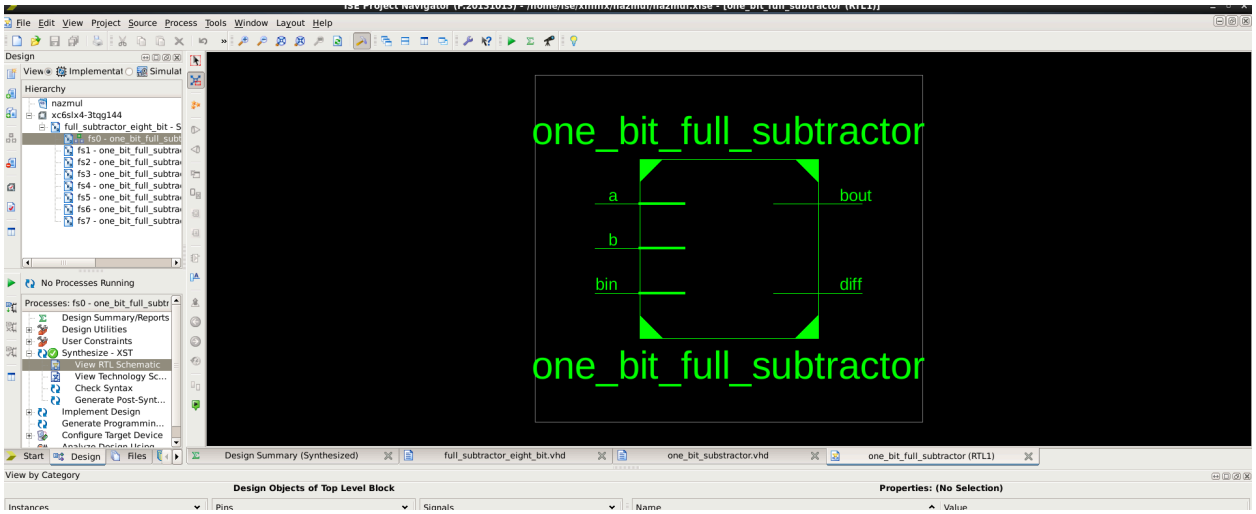
- Use VHDL and IEEE.STD_LOGIC_1164.ALL.
- The design should be structural/modular and should demonstrate the subtractor components covered in the laboratory.
- Do not use a single behavioral arithmetic statement such as D <= A - B; as the complete implementation.
- Create a separate testbench and generate appropriate input stimulus.
- Display A, B, and D in the simulation waveform.
- No clock or register is required for this task; the required circuit is combinational.

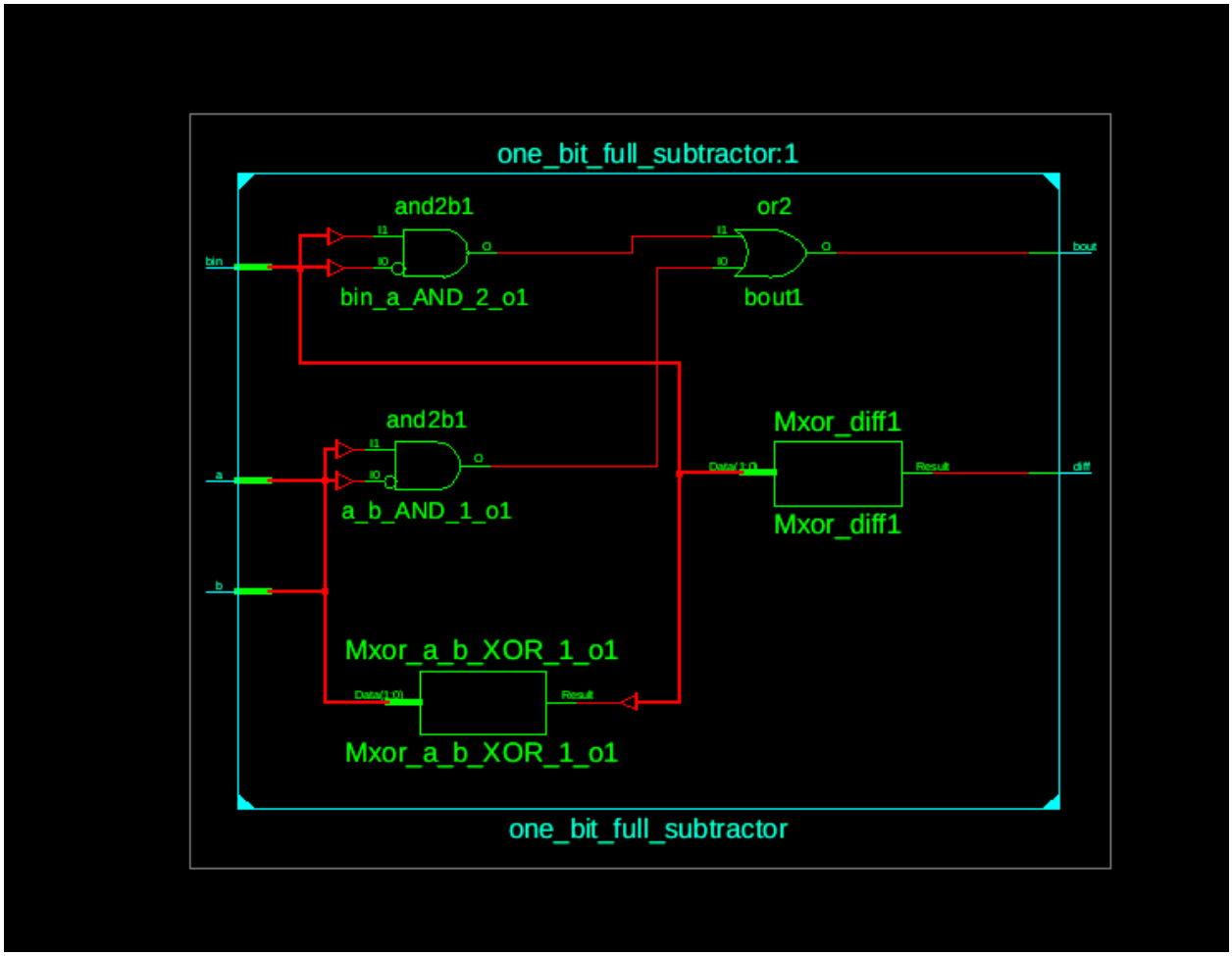
Submission

While submitting to the classroom prepare the report from the third page of this template including the current template in the first and second page. From the third page write your report including but not limited to the following points.

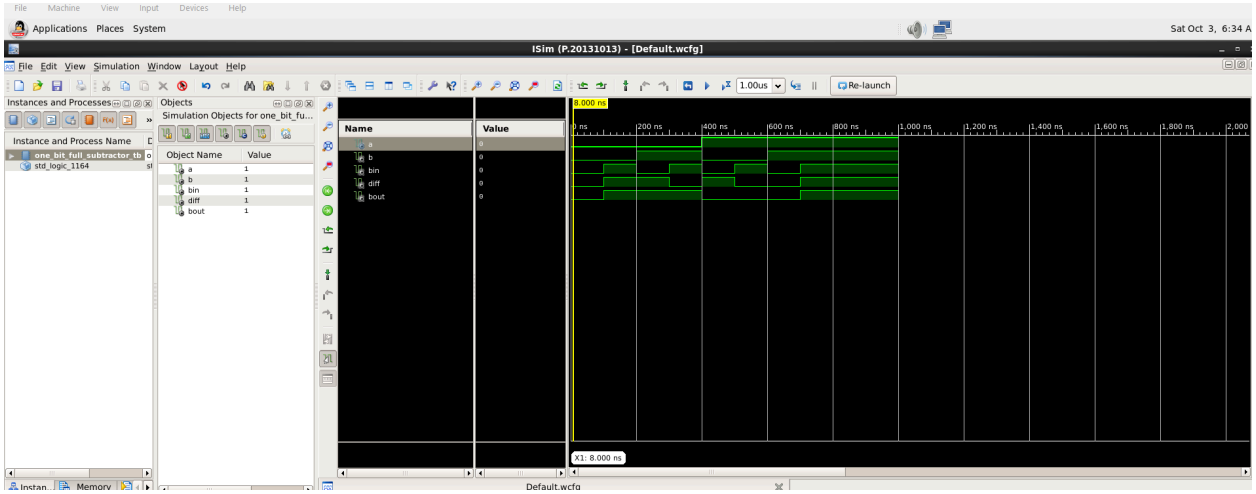
- Attach your entity code and testbench code
- Attach your output visuals
- Challenges faced during the experiment
- Conclusion

#one_bit_full_subtractor:

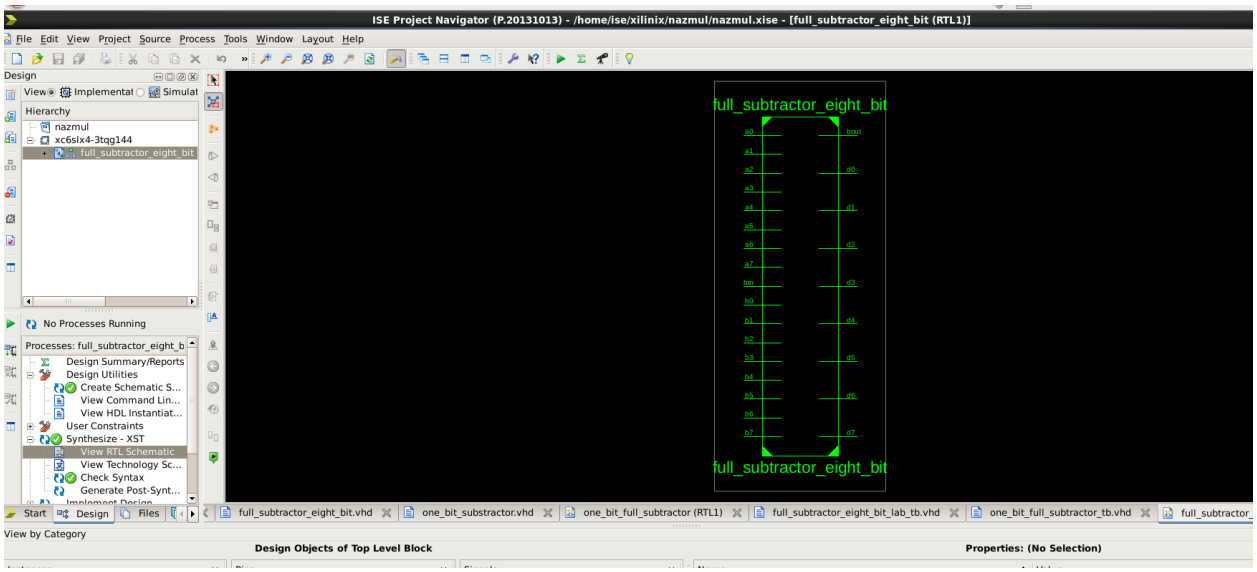


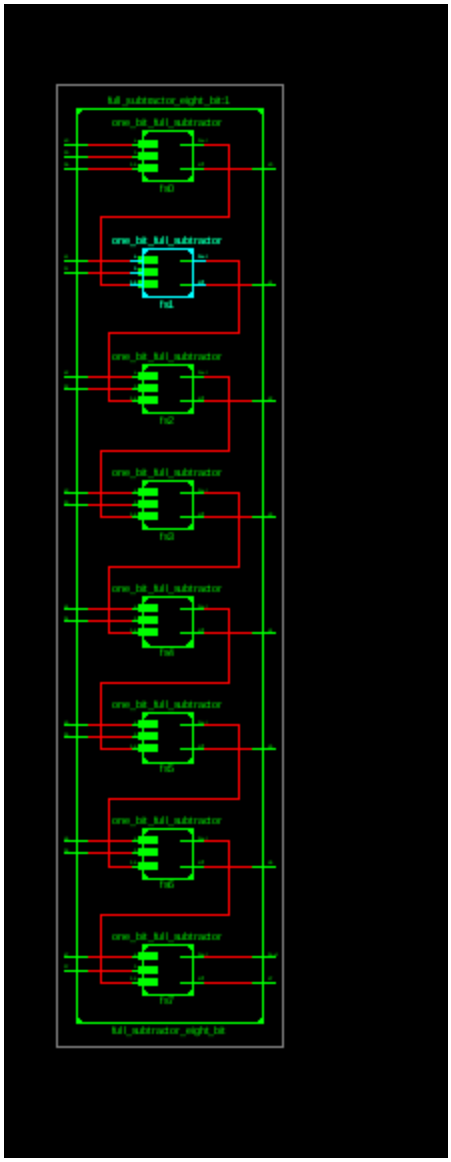


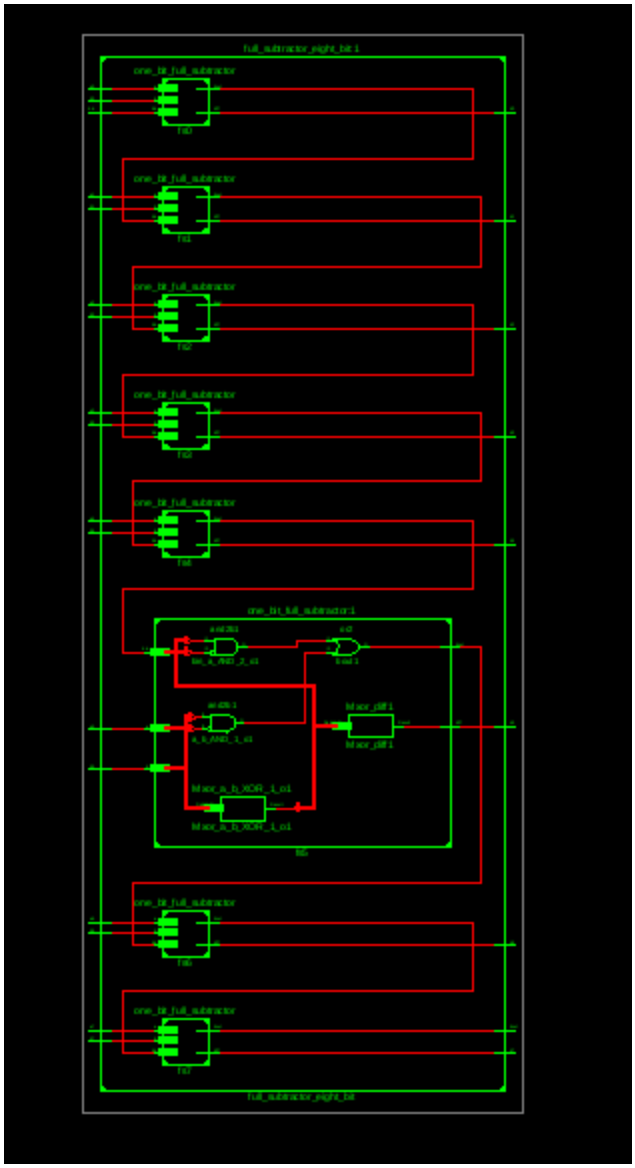
#one_bit_full_subtractor_tb:



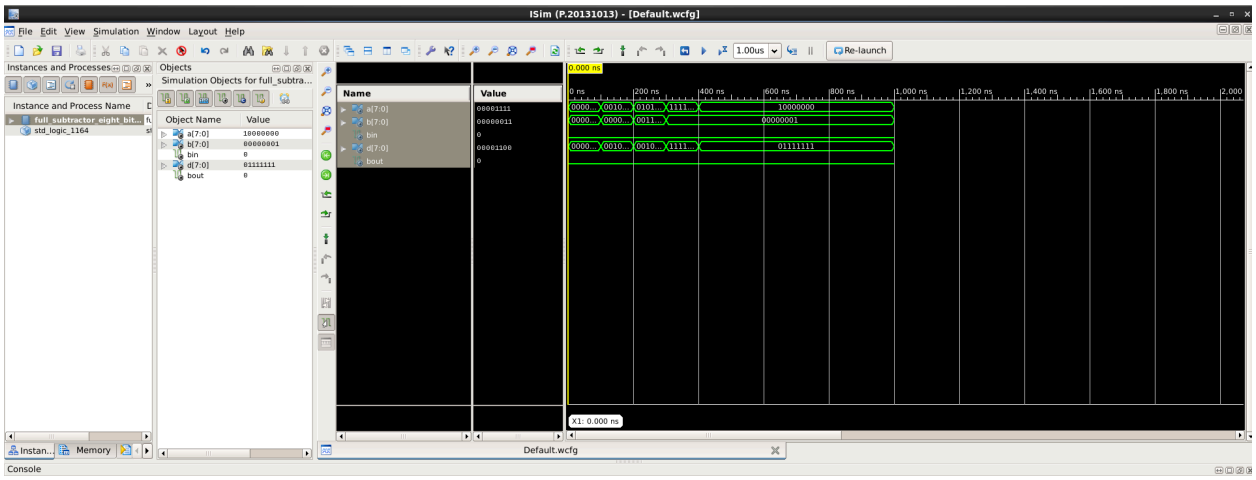
#full_subtractor_eight_bit:




full_subtractor_eight_bit
one_bit_full_subtractor
FA0
one_bit_full_subtractor
FA1
one_bit_full_subtractor
FA2
one_bit_full_subtractor
FA3
one_bit_full_subtractor
FA4
one_bit_full_subtractor
FA5
one_bit_full_subtractor
FA6
one_bit_full_subtractor
FA7
full_subtractor_eight_bit


full_subtractor_eight_bit
one_bit_full_subtractor
one_bit_full_subtractor
full_subtractor_eight_bit

#full_subtractor_eight_bit_tb:



#one_bit_full_subtractor:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- 1-bit full subtractor:  diff = a - b - bin
--   diff = a xor b xor bin
--   bout = 1 when a < b + bin  (a borrow is needed from the next bit)
entity one_bit_full_subtractor is
  Port ( a    : in  STD_LOGIC;
         b    : in  STD_LOGIC;
         bin  : in  STD_LOGIC;   -- borrow in
         diff : out STD_LOGIC;
         bout : out STD_LOGIC);  -- borrow out
end one_bit_full_subtractor;
architecture Behavioral of one_bit_full_subtractor is

begin
diff <= a xor b xor bin;
bout <= ((not a) and b) or (bin and (not (a xor b)));

end Behavioral;
```

#one_bit_full_subtractor_tb:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY one_bit_full_subtractor_tb IS
END one_bit_full_subtractor_tb;

ARCHITECTURE behavior OF one_bit_full_subtractor_tb IS

    -- Component Declaration for the Unit Under Test (UUT)
    COMPONENT one_bit_full_subtractor
    PORT(
                              a    : IN  std_logic;
         b    : IN  std_logic;
         bin  : IN  std_logic;
         diff : OUT std_logic;
         bout : OUT std_logic
        );
    END COMPONENT;
--Inputs
   signal a   : std_logic := '0';
   signal b   : std_logic := '0';
   signal bin : std_logic := '0';

   --Outputs
   signal diff : std_logic;
   signal bout : std_logic;

BEGIN
   -- Instantiate the Unit Under Test (UUT)
   uut: one_bit_full_subtractor PORT MAP (
          a => a,
          b => b,
          bin => bin,
          diff => diff,
          bout => bout
        );
```

```
  -- Stimulus process: all 8 input combinations, each result checked
  --   a b bin | diff bout
stim_proc: process
    procedure check(ia, ib, ibin, ediff, ebout : std_logic) is
    begin
      a <= ia; b <= ib; bin <= ibin;
      wait for 100 ns;
      assert (diff = ediff and bout = ebout)
        report "MISMATCH for a=" & std_logic'image(ia) &
               " b=" & std_logic'image(ib) &
               " bin=" & std_logic'image(ibin)
        severity error;
    end procedure;
  begin
    check('0','0','0', '0','0');   -- 0-0-0 = 0
    check('0','0','1', '1','1');   -- 0-0-1 = -1 -> diff 1, borrow
    check('0','1','0', '1','1');   -- 0-1-0 = -1 -> diff 1, borrow
    check('0','1','1', '0','1');   -- 0-1-1 = -2 -> diff 0, borrow
    check('1','0','0', '1','0');   -- 1-0-0 = 1
                  check('1','0','1', '0','0');   -- 1-0-1 = 0
    check('1','1','0', '0','0');   -- 1-1-0 = 0
    check('1','1','1', '1','1');   -- 1-1-1 = -1 -> diff 1, borrow
    report "one_bit_full_subtractor_tb finished" severity note;
    wait;
  end process;

END;
```

#full_subtractor_eight_bit:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- 8-bit ripple-borrow subtractor:  d = a - b - bin
-- bout = '1' when the result is negative (a < b + bin) as an unsigned subtraction.
entity full_subtractor_eight_bit is
   Port ( a0 : in  STD_LOGIC;
          a1 : in  STD_LOGIC;
          a2 : in  STD_LOGIC;
```

```
        a3 : in  STD_LOGIC;
                        a4 : in  STD_LOGIC;
        a5 : in  STD_LOGIC;
        a6 : in  STD_LOGIC;
        a7 : in  STD_LOGIC;
        b0 : in  STD_LOGIC;
        b1 : in  STD_LOGIC;
        b2 : in  STD_LOGIC;
                        b3 : in  STD_LOGIC;
        b4 : in  STD_LOGIC;
        b5 : in  STD_LOGIC;
        b6 : in  STD_LOGIC;
        b7 : in  STD_LOGIC;
        bin : in  STD_LOGIC;
        d0 : out  STD_LOGIC;
        d1 : out  STD_LOGIC;
        d2 : out  STD_LOGIC;
        d3 : out  STD_LOGIC;
        d4 : out  STD_LOGIC;
        d5 : out  STD_LOGIC;
        d6 : out  STD_LOGIC;
        d7 : out  STD_LOGIC;
        bout : out  STD_LOGIC);
end full_subtractor_eight_bit;
architecture Structural of full_subtractor_eight_bit is

 component one_bit_full_subtractor
    Port ( a : in  STD_LOGIC;
        b : in  STD_LOGIC;
        bin : in  STD_LOGIC;
        diff : out  STD_LOGIC;
        bout : out  STD_LOGIC);
  end component;
   signal br1,br2,br3,br4,br5,br6,br7 : STD_LOGIC;  -- borrow chain
begin
   fs0 : one_bit_full_subtractor port map (a=>a0,b=>b0,bin=>bin,diff=>d0,bout=>br1);
   fs1 : one_bit_full_subtractor port map (a=>a1,b=>b1,bin=>br1,diff=>d1,bout=>br2);
   fs2 : one_bit_full_subtractor port map (a=>a2,b=>b2,bin=>br2,diff=>d2,bout=>br3);
   fs3 : one_bit_full_subtractor port map (a=>a3,b=>b3,bin=>br3,diff=>d3,bout=>br4);
   fs4 : one_bit_full_subtractor port map (a=>a4,b=>b4,bin=>br4,diff=>d4,bout=>br5);
```

```
    fs5 : one_bit_full_subtractor port map (a=>a5,b=>b5,bin=>br5,diff=>d5,bout=>br6);
    fs6 : one_bit_full_subtractor port map (a=>a6,b=>b6,bin=>br6,diff=>d6,bout=>br7);
    fs7 : one_bit_full_subtractor port map (a=>a7,b=>b7,bin=>br7,diff=>d7,bout=>bout);

end Structural;
```

#full_subtractor_eight_bit_tb:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY full_subtractor_eight_bit_lab_tb IS
END full_subtractor_eight_bit_lab_tb;

ARCHITECTURE behavior OF full_subtractor_eight_bit_lab_tb IS

    -- Component Declaration for the Unit Under Test (UUT)
 COMPONENT full_subtractor_eight_bit
    PORT(
        a0 : IN  std_logic;
        a1 : IN  std_logic;
        a2 : IN  std_logic;
        a3 : IN  std_logic;
        a4 : IN  std_logic;
        a5 : IN  std_logic;
        a6 : IN  std_logic;
        a7 : IN  std_logic;
        b0 : IN  std_logic;
        b1 : IN  std_logic;
        b2 : IN  std_logic;
        b3 : IN  std_logic;
        b4 : IN  std_logic;
        b5 : IN  std_logic;
        b6 : IN  std_logic;
```

```
        b7 : IN  std_logic;
        bin : IN  std_logic;
        d0 : OUT  std_logic;
        d1 : OUT  std_logic;
        d2 : OUT  std_logic;
        d3 : OUT  std_logic;
        d4 : OUT  std_logic;
        d5 : OUT  std_logic;
        d6 : OUT  std_logic;
        d7 : OUT  std_logic;
        bout : OUT  std_logic
       );
 END COMPONENT;

   --Inputs
   signal A   : std_logic_vector(7 downto 0) := (others => '0');
   signal B   : std_logic_vector(7 downto 0) := (others => '0');
   signal bin : std_logic := '0';
--Outputs
   signal D    : std_logic_vector(7 downto 0);
   signal bout : std_logic;

   -- helper: std_logic_vector -> text, so the console log shows the bits
   function to_bits(v : std_logic_vector) return string is
     variable r : string(1 to v'length);
     variable k : integer := 1;
   begin
    for i in v'range loop
        if v(i) = '1' then r(k) := '1';
        elsif v(i) = '0' then r(k) := '0';
        else r(k) := 'X';
        end if;
        k := k + 1;
     end loop;
     return r;
   end function;

BEGIN

-- Instantiate the Unit Under Test (UUT)
```

```
  uut: full_subtractor_eight_bit PORT MAP (
        a0 => A(0), a1 => A(1), a2 => A(2), a3 => A(3),
        a4 => A(4), a5 => A(5), a6 => A(6), a7 => A(7),
        b0 => B(0), b1 => B(1), b2 => B(2), b3 => B(3),
        b4 => B(4), b5 => B(5), b6 => B(6), b7 => B(7),
        bin => bin,
        d0 => D(0), d1 => D(1), d2 => D(2), d3 => D(3),
        d4 => D(4), d5 => D(5), d6 => D(6), d7 => D(7),
        bout => bout
      );
-- Stimulus process
  stim_proc: process

     -- apply one test case, wait, then compare D with the expected value
     procedure run_test(n          : integer;
                        a_in       : std_logic_vector(7 downto 0);
                        b_in       : std_logic_vector(7 downto 0);
                        expected_d : std_logic_vector(7 downto 0)) is
begin
        A   <= a_in;
        B   <= b_in;
        bin <= '0';
        wait for 100 ns;

        if D = expected_d and bout = '0' then
          report "Test " & integer'image(n) &
                 ": A=" & to_bits(a_in) & " B=" & to_bits(b_in) &
                 " D=" & to_bits(D) & "  PASS";
        else
 report "Test " & integer'image(n) &
                 ": A=" & to_bits(a_in) & " B=" & to_bits(b_in) &
                 " D=" & to_bits(D) & " expected D=" & to_bits(expected_d) &
                 "  FAIL"
             severity error;
        end if;
     end procedure;

  begin
  -- Test Case 1: 15 - 3   -> 12
     run_test(1, "00001111", "00000011", "00001100");
```

```
    -- Test Case 2: 42 - 10  -> 32
    run_test(2, "00101010", "00001010", "00100000");

    -- Test Case 3: 85 - 51  -> 34
    run_test(3, "01010101", "00110011", "00100010");

    -- Test Case 4: 255 - 1  -> 254
    run_test(4, "11111111", "00000001", "11111110");
-- Test Case 5: 128 - 1  -> 127
    run_test(5, "10000000", "00000001", "01111111");

    report "All 5 required test cases applied." severity note;
    wait;
  end process;

END;
```

Challenges:

- Getting the borrow equation correct took some care.
- Connecting the borrow from one stage to the next was important.
- Wiring all the input and output ports without mixing up the bit order was challenging.
- The files had to be added in the correct order so the components worked properly.
- I also had to make sure the testbench showed the 8-bit values clearly and matched the expected results.

Conclusion:

I built a 1-bit full subtractor and used eight of them to make an 8-bit subtractor. I did not use D <= A - B. All five test cases gave the correct results, such as 15 − 3 = 00001100 and 128 − 1 = 01111111. Since there is no clock, the circuit is combinational.

This lab helped me understand how a larger circuit can be built using smaller reusable components.